

3.2 Singly Linked Lists

In the previous section, we presented the array data structure and discussed some of its applications. Arrays are great for storing things in a certain order, but they have drawbacks. The capacity of the array must be fixed when it is created, and insertions and deletions at interior positions of an array can be time consuming if many elements must be shifted.

In this section, we introduce a data structure known as a **linked list**, which provides an alternative to an array-based structure. A linked list, in its simplest form, is a collection of **nodes** that collectively form a linear sequence. In a **singly linked list**, each node stores a reference to an object that is an element of the sequence, as well as a reference to the next node of the list (see Figure 3.10).

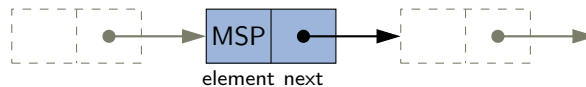


Figure 3.10: Example of a node instance that forms part of a singly linked list. The node's element field refers to an object that is an element of the sequence (the airport code MSP, in this example), while the next field refers to the subsequent node of the linked list (or null if there is no further node).

A linked list's representation relies on the collaboration of many objects (see Figure 3.11). Minimally, the linked list instance must keep a reference to the first node of the list, known as the **head**. Without an explicit reference to the head, there would be no way to locate that node (or indirectly, any others). The last node of the list is known as the **tail**. The tail of a list can be found by **traversing** the linked list—starting at the head and moving from one node to another by following each node's next reference. We can identify the tail as the node having **null** as its next reference. This process is also known as **link hopping** or **pointer hopping**. However, storing an explicit reference to the tail node is a common efficiency to avoid such a traversal. In similar regard, it is common for a linked list instance to keep a count of the total number of nodes that comprise the list (also known as the **size** of the list), to avoid traversing the list to count the nodes.

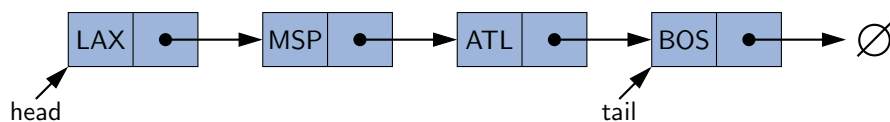


Figure 3.11: Example of a singly linked list whose elements are strings indicating airport codes. The list instance maintains a member named **head** that refers to the first node of the list, and another member named **tail** that refers to the last node of the list. The **null** value is denoted as $\emptyset$.

Inserting an Element at the Head of a Singly Linked List

An important property of a linked list is that it does not have a predetermined fixed size; it uses space proportional to its current number of elements. When using a singly linked list, we can easily insert an element at the head of the list, as shown in Figure 3.12, and described with pseudocode in Code Fragment 3.11. The main idea is that we create a new node, set its element to the new element, set its next link to refer to the current head, and set the list's head to point to the new node.

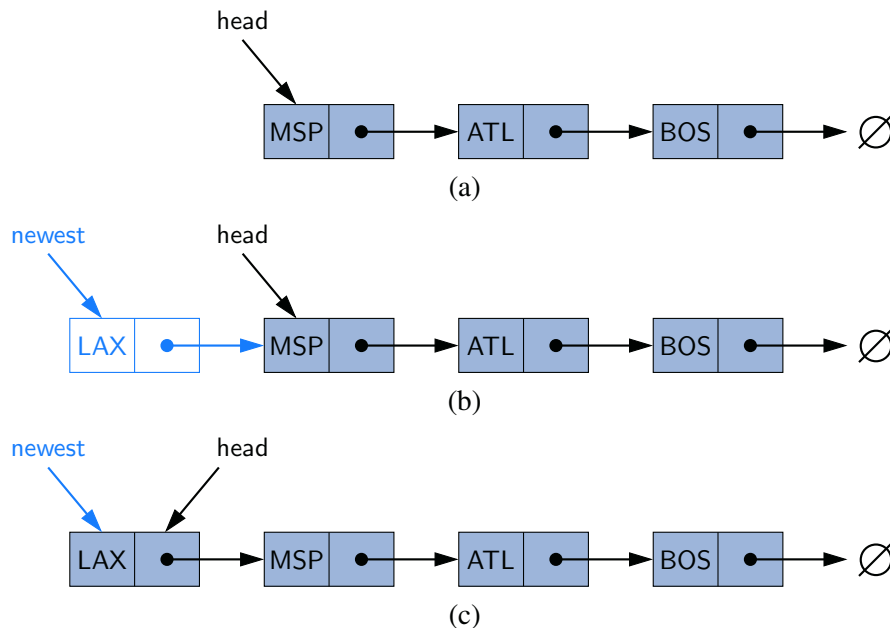


Figure 3.12: Insertion of an element at the head of a singly linked list: (a) before the insertion; (b) after a new node is created and linked to the existing head; (c) after reassignment of the head reference to the newest node.

Algorithm addFirst(e):

```

newest = Node( $e$ )  {create new node instance storing reference to element  $e$ }
newest.next = head {set new node's next to reference the old head node}
head = newest      {set variable head to reference the new node}
size = size + 1   {increment the node count}

```

Code Fragment 3.11: Inserting a new element at the beginning of a singly linked list. Note that we set the next pointer of the new node *before* we reassign variable **head** to it. If the list were initially empty (i.e., **head** is null), then a natural consequence is that the new node has its next reference set to null.

Inserting an Element at the Tail of a Singly Linked List

We can also easily insert an element at the tail of the list, provided we keep a reference to the tail node, as shown in Figure 3.13. In this case, we create a new node, assign its next reference to null, set the next reference of the tail to point to this new node, and then update the tail reference itself to this new node. We give pseudocode for the process in Code Fragment 3.12.

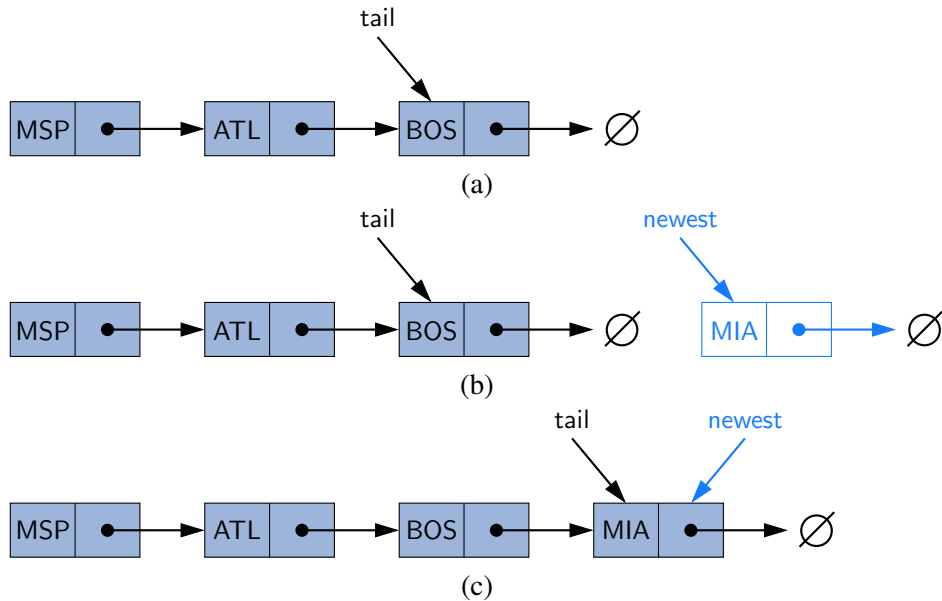


Figure 3.13: Insertion at the tail of a singly linked list: (a) before the insertion; (b) after creation of a new node; (c) after reassignment of the tail reference. Note that we must set the next link of the tail node in (b) before we assign the tail variable to point to the new node in (c).

Algorithm addLast(e):

```

newest = Node( $e$ )    {create new node instance storing reference to element  $e$ }
newest.next = null   {set new node's next to reference the null object}
tail.next = newest    {make old tail node point to new node}
tail = newest         {set variable tail to reference the new node}
size = size + 1      {increment the node count}

```

Code Fragment 3.12: Inserting a new node at the end of a singly linked list. Note that we set the next pointer for the old tail node *before* we make variable tail point to the new node. This code would need to be adjusted for inserting onto an empty list, since there would not be an existing tail node.

Removing an Element from a Singly Linked List

Removing an element from the *head* of a singly linked list is essentially the reverse operation of inserting a new element at the head. This operation is illustrated in Figure 3.14 and described in detail in Code Fragment 3.13.

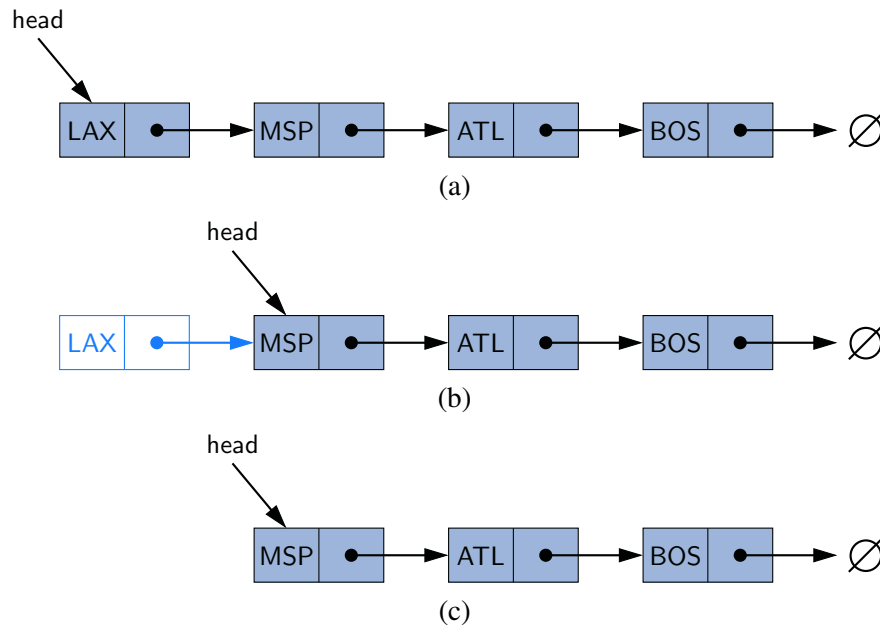


Figure 3.14: Removal of an element at the head of a singly linked list: (a) before the removal; (b) after “linking out” the old head; (c) final configuration.

Algorithm `removeFirst()`:

if `head == null` **then**

 the list is empty.

`head = head.next`

`size = size - 1`

 {make head point to next node (or null)}

 {decrement the node count}

Code Fragment 3.13: Removing the node at the beginning of a singly linked list.

Unfortunately, we cannot easily delete the last node of a singly linked list. Even if we maintain a tail reference directly to the last node of the list, we must be able to access the node *before* the last node in order to remove the last node. But we cannot reach the node before the tail by following next links from the tail. The only way to access this node is to start from the head of the list and search all the way through the list. But such a sequence of link-hopping operations could take a long time. If we want to support such an operation efficiently, we will need to make our list *doubly linked* (as we do in Section 3.4).

3.2.1 Implementing a Singly Linked List Class

In this section, we present a complete implementation of a `SinglyLinkedList` class, supporting the following methods:

- `size()`: Returns the number of elements in the list.
- `isEmpty()`: Returns **true** if the list is empty, and **false** otherwise.
- `first()`: Returns (but does not remove) the first element in the list.
- `last()`: Returns (but does not remove) the last element in the list.
- `addFirst(e)`: Adds a new element to the front of the list.
- `addLast(e)`: Adds a new element to the end of the list.
- `removeFirst()`: Removes and returns the first element of the list.

If `first()`, `last()`, or `removeFirst()` are called on a list that is empty, we will simply return a **null** reference and leave the list unchanged.

Because it does not matter to us what type of elements are stored in the list, we use Java's *generics framework* (see Section 2.5.2) to define our class with a formal type parameter *E* that represents the user's desired element type.

Our implementation also takes advantage of Java's support for *nested classes* (see Section 2.6), as we define a private `Node` class within the scope of the public `SinglyLinkedList` class. Code Fragment 3.14 presents the `Node` class definition, and Code Fragment 3.15 the rest of the `SinglyLinkedList` class. Having `Node` as a nested class provides strong encapsulation, shielding users of our class from the underlying details about nodes and links. This design also allows Java to differentiate this node type from forms of nodes we may define for use in other structures.

```

1 public class SinglyLinkedList<E> {
2     //----- nested Node class -----
3     private static class Node<E> {
4         private E element;           // reference to the element stored at this node
5         private Node<E> next;       // reference to the subsequent node in the list
6         public Node(E e, Node<E> n) {
7             element = e;
8             next = n;
9         }
10        public E getElement() { return element; }
11        public Node<E> getNext() { return next; }
12        public void setNext(Node<E> n) { next = n; }
13    } //----- end of nested Node class -----
    ... rest of SinglyLinkedList class will follow ...

```

Code Fragment 3.14: A nested `Node` class within the `SinglyLinkedList` class. (The remainder of the `SinglyLinkedList` class will be given in Code Fragment 3.15.)

```

1  public class SinglyLinkedList<E> {
...  (nested Node class goes here)

14  // instance variables of the SinglyLinkedList
15  private Node<E> head = null;           // head node of the list (or null if empty)
16  private Node<E> tail = null;          // last node of the list (or null if empty)
17  private int size = 0;                  // number of nodes in the list
18  public SinglyLinkedList() { }           // constructs an initially empty list
19  // access methods
20  public int size() { return size; }
21  public boolean isEmpty() { return size == 0; }
22  public E first() {                     // returns (but does not remove) the first element
23      if (isEmpty()) return null;
24      return head.getElement();
25  }
26  public E last() {                       // returns (but does not remove) the last element
27      if (isEmpty()) return null;
28      return tail.getElement();
29  }
30  // update methods
31  public void addFirst(E e) {              // adds element e to the front of the list
32      head = new Node<>(e, head);          // create and link a new node
33      if (size == 0)
34          tail = head;                    // special case: new node becomes tail also
35      size++;
36  }
37  public void addLast(E e) {              // adds element e to the end of the list
38      Node<E> newest = new Node<>(e, null); // node will eventually be the tail
39      if (isEmpty())
40          head = newest;                   // special case: previously empty list
41      else
42          tail.setNext(newest);            // new node after existing tail
43      tail = newest;                       // new node becomes the tail
44      size++;
45  }
46  public E removeFirst() {                // removes and returns the first element
47      if (isEmpty()) return null;          // nothing to remove
48      E answer = head.getElement();
49      head = head.getNext();              // will become null if list had only one node
50      size--;
51      if (size == 0)
52          tail = null;                    // special case as list is now empty
53      return answer;
54  }
55  }

```

Code Fragment 3.15: The SinglyLinkedList class definition (when combined with the nested Node class of Code Fragment 3.14).